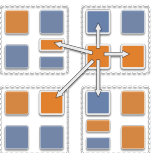


Asynchronous Methods

- Entry methods are invoked by performing a Python method call on a chore object

```
c = Chore(Hello)
c.sayHi(17)
```

- The policy is ***one-at-a-time scheduling*** (that is, one entry method on one chore executes on a processor at a time)



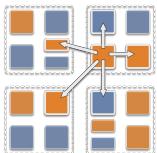
Asynchronous Methods

- Method invocation is not ordered (between chares, entry methods on one chare, etc.)!
- For example, if a chare executes this code:

```
c = Chare(Hello)
c.foo()
c.bar()
```

- These prints may occur in **any** order

```
def foo(self):
    print("foo executes")
def bar(self, param):
    print("bar executes with ", param)
```



Asynchronous Methods

- If a chore invokes the same entry method twice

```
c.bar(7)  
c.bar(5)
```

- These may be delivered in **any** order

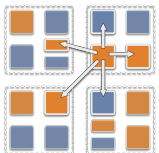
```
def bar(self, param):  
    print("bar executes with ", param)
```

- Output

```
bar executes with 5  
bar executes with 7
```

or

```
bar executes with 7  
bar executes with 5
```



Does this program execute correctly?

```
class Simple(Chare):
    def __init__(self, pi):
        self.y = pi
    def findArea(self, r, done):
        print("radius: " + r + " Area: " + y*r*r)
        if done:
            exit()

def main(args):
    sim = Chare(Simple, args=[3.1415])
    for i in range(10):
        sim.findArea(i, False)
    sim.findArea(10, True)
```

No! Methods are Asynchronous

- If a chore invokes several entry methods

```
sim.findArea(1, false)
...
sim.findArea(10, true)
```

- These may be delivered in **any** order

```
def findArea(self, r, done):
    print("radius: " + r + " Area: " +
          y*r*r)
    if count++ = 10:
        exit()
```

- Output:

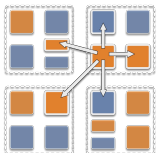
```
radius: 254.34
radius: 200.96
radius: 28.26
radius: 3.14
radius: 12.56
radius: 153.86
radius: 50.24
radius: 78.50
radius: 314.00
```

or

```
radius: 28.26
radius: 78.50
radius: 3.14
radius: 113.04
radius: 314.00
```

Readonly Variables

- The only global variables officially allowed in a Charm++ program are readonly variables
 - readonly can be modified only in the constructor of the main charc
 - And from any functions called from them (not entry methods)
 - After the constructor finishes, before any other charcs execute any methods, the readonly variables are available on all processors



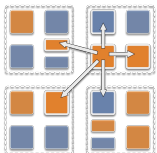
Data Types and Entry Methods

- You can pass basic C++ types to entry methods (int, char, bool)
- C++ STL data structures can be passed by including `pup_stl.h`
- Arrays of basic data types can also be passed like this:
- `.ci` file:

```
entry void foobar(int length, int data[length]);
```

- `.cpp` file

```
MyChare::foobar(int length, int* data) {  
    // ... foobar code ...  
}
```



Exercise 1. Fibonacci Sequence

- Files under **exercises/ex1**
- **Goal:** Learn how to compile and run a Charm++ program
 - A simple, recursive program that computes a Fibonacci number
 - Detailed instructions provided in README
- Please let us know if you run into any problems!

